DOMAIN 3

PROMPT ENGINEERING & STRUCTURED OUTPUT

Complete deep-dive for this blueprint domain (20% of exam) of the Claude Certified Architect — Foundations exam: every concept with worked examples, a dedicated traps-and-nuances section explaining why the tempting wrong answers are wrong, a one-page cheat sheet, and a full bank of 118 original practice questions with answers and rationales.

Section	What's inside
1. Cheat Sheet	Golden rule, quick-reference tables, and a scenario decision checklist — read this first and last.
2. Concept Guide	Full narrative coverage of every concept tested in this domain, with worked scenarios.
3. Traps & Nuances	16 recurring wrong-answer patterns: the trap, why it's tempting, and the correct principle.
4. Practice Questions	118 original scenario-based questions, 4 options each, answer marked, rationale included.

This is an independent study aid, not an official Anthropic publication or a reproduction of actual exam questions, which Anthropic does not publish.

1 · CHEAT SHEET

Golden rule: schemas and tool_use guarantee syntax (valid shape/types), never semantics (factual or arithmetic correctness). Almost every Domain 3 trap confuses “validated” with “correct.”

Schema field design

Required field	Only if the data is genuinely ALWAYS present in the source.
Nullable field	type: ["string","null"] for anything that may legitimately be absent.
Enum design	Always include "unclear" / "other" as an escape hatch.

Few-shot targeting

Target	Ambiguous / boundary cases — where the model actually errs.
Not this	Easy, already-reliable cases — low marginal value.
Include	The rationale behind each labeled example, not just the answer.

Retry-with-feedback: works vs. doesn't

Works	Data exists in the source but was mis-extracted or miscalculated.
Doesn't work	Data was never in the source at all — flag the gap instead.

Batch API fit test

Good fit	~50% cheaper, up to 24h window, NO latency SLA — nothing/no one blocking on it.
Poor fit	Anything synchronous, user-facing, or blocking (pre-merge checks, live chat, real-time gating).

Decision checklist for any Domain 3 scenario

- Output is valid JSON but still wrong? → semantic error — add a validation step, don't blame the schema.
- Field sometimes missing from source? → make it nullable, not required.
- Classifier keeps forcing bad fits into categories? → add “unclear”/“other,” don't add more categories.
- Retry loop not fixing a field? → check whether the data is in the source at all before retrying further.
- Considering Batch API? → ask “is anyone waiting in real time” first, cost savings second.
- Need consistent structured output? → lower temperature + targeted few-shot examples.

DOMAIN 3 20% of exam

Prompt Engineering & Structured Output

2 · CONCEPT GUIDE

The exam assumes baseline prompting competence and instead probes what happens at production scale: schema design, the difference between syntactic and semantic correctness, few-shot targeting, retry strategy limits, and when the Batch API is (and isn't) appropriate.

3.1 tool_use for structured output

Defining a JSON Schema and forcing the model to respond through a tool call (rather than free text you parse yourself) eliminates an entire class of failures: malformed JSON, missing braces, trailing commas, inconsistent key names. What it does **not** eliminate is **semantic** error — output that is perfectly valid JSON and still wrong. Two examples the exam likes:

- A **stated_total** field that doesn't actually match the sum of the **line_items** array — syntactically fine, arithmetically wrong.
- A required field populated with a **fabricated value** because the source document had no data for it, and the schema forced the model to supply *something*.

The exam's core lesson here: schema validation guarantees shape, not truth. You still need downstream checks (arithmetic validation, cross-field consistency checks, source-grounding checks) for correctness.

3.2 Schema design rules

- Mark a field **required** only if the information is genuinely always present in the input — a required field with no source data pressures the model to invent a plausible-looking value rather than surface the gap.
- Use **type: ["string", "null"]** (a nullable type) for fields that may legitimately be absent, so the model can express “not present” instead of guessing.
- For classification enums, add explicit **"unclear"** and **"other"** values rather than forcing every input into one of a fixed set of categories that might not fit — this is a common exam distractor: an enum with no escape-hatch value silently corrupts the data whenever a real input doesn't cleanly match any listed category.

3.3 Few-shot examples

For consistent structured output, well-chosen few-shot examples generally outperform longer descriptive instructions. The exam tests **targeting**: examples are most valuable when aimed at the specific cases the model actually gets wrong — ambiguous inputs, boundary cases, categories that overlap — rather than at the easy, already-reliable cases. Include the **rationale** behind each example's labeled answer, not just the answer itself, so the pattern the model should generalize is visible, not just memorized.

3.4 Retry-with-feedback: when it works and when it doesn't

A retry loop that feeds validation errors back to the model (“your total didn't match the line items, try again”) is effective when the correct information genuinely exists somewhere in the input and the first pass simply mis-extracted or miscalculated it. It is **not** effective when the needed information isn't in the source document at all — no amount of retrying recovers data that was never provided. The exam uses this distinction to test whether you'll build a retry pipeline for a problem retrying structurally cannot fix; the correct move in that case is a different one — flag the gap, request the

missing source, or route to a human.

3.5 Batch API — when it fits and when it doesn't

The Batch API offers roughly 50% cost savings against standard pricing, with results returned within an up-to-24-hour window and **no latency SLA** — Anthropic makes no commitment about when within that window a given batch will complete.

Good fit	Poor fit
Overnight reports	Blocking pre-merge CI checks
Large-scale bulk classification/extraction	Anything a user is actively waiting on
Periodic tech-debt / analytics sweeps	Real-time chat or support interactions

Worked scenario

A manager proposes moving two workloads to the Batch API to cut cost: a blocking pre-merge code check, and a nightly technical-debt report. Correct answer: move the **nightly report** to Batch (it fits the overnight, non-blocking profile) and keep the **pre-merge check synchronous** — a developer sitting at their terminal cannot be told to wait up to 24 hours with no SLA before they can merge.

3.6 Summary: syntactic vs. semantic correctness

- **Syntactic correctness** (schemas / tool_use solve this): valid JSON, correct types, required fields present, valid enum values.
- **Semantic correctness** (schemas do NOT solve this): numbers that don't add up, fabricated values, internally inconsistent fields, claims not grounded in the source — needs explicit validation logic or a verification pass.

3 · TRAPS & NUANCES

Each card below names a specific concept, describes the wrong answer the exam is likely to dangle in front of you, explains why that wrong answer feels right, and states the principle that actually resolves it. Read this section right before the exam as a final refresh.

CONCEPT: tool_use / schema-constrained output

The trap: Treating a response that validates against the JSON schema as automatically correct.

Why it's tempting: Passing validation feels like a pass/fail correctness check, similar to a test suite.

The correct principle: Schemas guarantee syntactic validity (shape, types) only — a response can be perfectly valid JSON and still be arithmetically or factually wrong.

CONCEPT: Required schema fields

The trap: Marking every field required “to make sure nothing gets missed.”

Why it's tempting: Required feels stricter and safer, like it forces more complete output.

The correct principle: A required field with no guaranteed source data pressures the model to invent a plausible-looking value — fields that may genuinely be absent should be nullable, not required.

CONCEPT: Enum design

The trap: Responding to enum mismatches by adding more specific categories instead of an escape-hatch value.

Why it's tempting: More categories feels like it increases precision and coverage.

The correct principle: The fix for enums that don't cover every real input is “unclear”/“other” values, not an ever-expanding list of specific categories that will still eventually miss something.

CONCEPT: Few-shot example targeting

The trap: Adding more few-shot examples of cases the model already classifies correctly.

Why it's tempting: “More examples = better performance” is a natural, general intuition about few-shot prompting.

The correct principle: Examples are most valuable when targeted at the model's actual failure cases — ambiguous or boundary inputs — not at cases it already handles well.

CONCEPT: Retry-with-feedback scope

The trap: Assuming enough retries will eventually surface a field that's missing from the source document entirely.

Why it's tempting: Retrying has worked before for fixing extraction mistakes, so it's tempting to apply it universally.

The correct principle: Retry-with-feedback only helps when the correct data exists in the input but was mis-extracted — it cannot recover information that was never present in the source, no matter how many attempts.

CONCEPT: Batch API fit

The trap: Choosing the Batch API purely because of the ~50% cost savings, without checking whether anyone is waiting on the result.

Why it's tempting: A clearly quantified cost saving is an easy, attractive number to optimize toward.

The correct principle: Batch API fit hinges on latency tolerance: it has no latency SLA and up to a 24-hour window, making it wrong for anything blocking or user-facing regardless of the cost benefit.

CONCEPT: Temperature for structured tasks

The trap: Using a higher temperature for structured extraction to make the model “try harder” or be more thorough.

Why it's tempting: Higher temperature is associated with more creative, effortful-sounding output in general use.

The correct principle: Deterministic, structured tasks benefit from lower temperature — consistency matters more than creative variation for extraction or classification.

CONCEPT: Prompt caching

The trap: Re-sending a large, unchanged context block on every call “to be safe” or keep it fresh.

Why it's tempting: Caching can sound risky, as if it might serve stale or outdated content.

The correct principle: Caching a genuinely stable, unchanged block doesn't create staleness — it reuses identical content and cuts both cost and latency; the risk framing only applies to content that actually changes.

CONCEPT: Structural tags in prompts

The trap: Assuming visually clear formatting (line breaks, paragraphs) is enough to separate instructions, context, and input for the model.

Why it's tempting: It reads clearly to a human, so it's easy to assume it reads just as clearly to the model.

The correct principle: Explicit structural tags around distinct sections give the model a much more reliable way to distinguish instructions from context from input than visual formatting alone.

CONCEPT: Reducing fabrication

The trap: Fixing hallucinated claims by instructing the model to “be more careful” or “double-check.”

Why it's tempting: It directly addresses the symptom in plain language, which feels like it should help.

The correct principle: Vague carefulness instructions are unreliable; requiring explicit grounding — tying each claim to a specific source reference — is the structural fix that actually reduces fabrication.

CONCEPT: System prompt vs. user turn placement

The trap: Repeating a stable behavioral rule in every user message “to reinforce it.”

Why it's tempting: Repetition feels like it should strengthen adherence to the rule.

The correct principle: Stable, task-wide behavior belongs once in the system prompt; repeating it per-turn is redundant and makes prompts harder to maintain without improving reliability.

CONCEPT: Evaluating prompt changes

The trap: Judging a prompt change by reading it and deciding it “looks right” before shipping.

Why it's tempting: A prompt is text, and reading text for correctness feels like a natural, sufficient review method.

The correct principle: Prompt changes should be measured against a held-out evaluation set with known-correct answers — how a prompt reads is a poor predictor of how it actually performs.

CONCEPT: Streaming structured output

The trap: Acting on a structured (JSON) response as soon as the first streamed tokens arrive.

Why it's tempting: Streaming implies “use data as it comes,” which is the normal mental model for streamed text.

The correct principle: Partially-streamed JSON isn't valid until the object is complete — downstream consumers need to wait for completion or use a schema-aware incremental parser.

CONCEPT: Cost optimization

The trap: Reaching for a different (often larger) model as the default fix when per-call cost is too high.

Why it's tempting: Model choice feels like the single biggest lever over cost.

The correct principle: Trimming unnecessary prompt content is usually the highest-leverage, lowest-risk cost optimization to try before changing models or infrastructure.

CONCEPT: Uncertain extractions

The trap: Silently dropping a field from the output when the model is genuinely uncertain about its value.

Why it's tempting: Omitting an uncertain value can feel safer than potentially reporting something wrong.

The correct principle: Uncertainty should be surfaced explicitly (e.g., a confidence field) so downstream systems can route it for review — silent omission just hides the gap instead of handling it.

CONCEPT: Schema evolution

The trap: Changing a shipped schema in place (renaming fields, changing types) as the direct fix for a design flaw.

Why it's tempting: Changing it directly is the fastest way to “fix” the schema.

The correct principle: Breaking schema changes need explicit versioning and a transition period — changing in place silently breaks every existing downstream consumer.

4 · PRACTICE QUESTIONS (118)

Q1. An invoice extractor uses tool_use with a strict JSON schema, and output is always valid JSON. Sometimes it still produces a stated_total field that doesn't match the sum of line_items. What does this reveal about schema-constrained output?

A. It's impossible for a schema-constrained response to contain this kind of error.

B. This means the schema itself is malformed and must be rejected by the API.

C. This only happens when tool_use is used without any temperature setting.

D. ✓ It only guarantees valid JSON shape and types — it can still return a semantically wrong, internally inconsistent value like this.

Correct answer: D. Schemas (via tool_use) guarantee syntactic validity — correct shape and types — but say nothing about whether the content is factually or arithmetically consistent; that needs a separate validation step.

Q2. A contract summarizer uses tool_use with a strict JSON schema, and output is always valid JSON. Sometimes it still produces a stated effective_date that contradicts a date mentioned elsewhere in the same contract. What does this reveal about schema-constrained output?

A. This means the schema itself is malformed and must be rejected by the API.

B. This only happens when tool_use is used without any temperature setting.

C. ✓ It only guarantees valid JSON shape and types — it can still return a semantically wrong, internally inconsistent value like this.

D. It's impossible for a schema-constrained response to contain this kind of error.

Correct answer: C. Schemas (via tool_use) guarantee syntactic validity — correct shape and types — but say nothing about whether the content is factually or arithmetically consistent; that needs a separate validation step.

Q3. A resume parser uses tool_use with a strict JSON schema, and output is always valid JSON. Sometimes it still produces a total_years_experience field that doesn't match the sum of the listed job durations. What does this reveal about schema-constrained output?

A. This only happens when tool_use is used without any temperature setting.

B. ✓ It only guarantees valid JSON shape and types — it can still return a semantically wrong, internally inconsistent value like this.

C. It's impossible for a schema-constrained response to contain this kind of error.

D. This means the schema itself is malformed and must be rejected by the API.

Correct answer: B. Schemas (via tool_use) guarantee syntactic validity — correct shape and types — but say nothing about whether the content is factually or arithmetically consistent; that needs a separate validation step.

Q4. An order-processing extractor uses tool_use with a strict JSON schema, and output is always valid JSON. Sometimes it still produces a shipping_cost field that's inconsistent with the stated shipping method. What does this reveal about schema-constrained output?

A. ✓ It only guarantees valid JSON shape and types — it can still return a semantically wrong, internally inconsistent value like this.

B. It's impossible for a schema-constrained response to contain this kind of error.

C. This means the schema itself is malformed and must be rejected by the API.

D. This only happens when tool_use is used without any temperature setting.

Correct answer: A. Schemas (via tool_use) guarantee syntactic validity — correct shape and types — but say nothing about whether the content is factually or arithmetically consistent; that needs a separate validation step.

Q5. A financial-report summarizer uses tool_use with a strict JSON schema, and output is always valid JSON. Sometimes it still produces a net_income field that doesn't match revenue minus the listed expenses. What does this reveal about schema-constrained output?

A. It's impossible for a schema-constrained response to contain this kind of error.

B. This means the schema itself is malformed and must be rejected by the API.

C. This only happens when tool_use is used without any temperature setting.

D. ✓ It only guarantees valid JSON shape and types — it can still return a semantically wrong, internally inconsistent value like this.

Correct answer: D. Schemas (via tool_use) guarantee syntactic validity — correct shape and types — but say nothing about whether the content is factually or arithmetically consistent; that needs a separate validation step.

Q6. A medical-form extractor uses tool_use with a strict JSON schema, and output is always valid JSON. Sometimes it still produces a patient_age that's inconsistent with the stated date of birth. What does this reveal about schema-constrained output?

A. This means the schema itself is malformed and must be rejected by the API.

B. This only happens when tool_use is used without any temperature setting.

C. ✓ It only guarantees valid JSON shape and types — it can still return a semantically wrong, internally inconsistent value like this.

D. It's impossible for a schema-constrained response to contain this kind of error.

Correct answer: C. Schemas (via tool_use) guarantee syntactic validity — correct shape and types — but say nothing about whether the content is factually or arithmetically consistent; that needs a separate validation step.

Q7. A real-estate listing parser uses tool_use with a strict JSON schema, and output is always valid JSON. Sometimes it still produces a price_per_sqft field that doesn't match price divided by square footage. What does this reveal about schema-constrained output?

A. This only happens when tool_use is used without any temperature setting.

B. ✓ It only guarantees valid JSON shape and types — it can still return a semantically wrong, internally inconsistent value like this.

C. It's impossible for a schema-constrained response to contain this kind of error.

D. This means the schema itself is malformed and must be rejected by the API.

Correct answer: B. Schemas (via tool_use) guarantee syntactic validity — correct shape and types — but say nothing about whether the content is factually or arithmetically consistent; that needs a separate validation step.

Q8. A survey-response classifier uses tool_use with a strict JSON schema, and output is always valid JSON. Sometimes it still produces a sentiment_score that contradicts the plain meaning of the quoted response text. What does this reveal about schema-constrained output?

A. ✓ It only guarantees valid JSON shape and types — it can still return a semantically wrong, internally inconsistent value like this.

B. It's impossible for a schema-constrained response to contain this kind of error.

C. This means the schema itself is malformed and must be rejected by the API.

D. This only happens when tool_use is used without any temperature setting.

Correct answer: A. Schemas (via tool_use) guarantee syntactic validity — correct shape and types — but say nothing about whether the content is factually or arithmetically consistent; that needs a separate validation step.

Q9. A legal-clause extractor uses tool_use with a strict JSON schema, and output is always valid JSON. Sometimes it still produces a governing_law field that doesn't match the jurisdiction named elsewhere in the document. What does this reveal about schema-constrained output?

A. It's impossible for a schema-constrained response to contain this kind of error.

B. This means the schema itself is malformed and must be rejected by the API.

C. This only happens when tool_use is used without any temperature setting.

D. ✓ It only guarantees valid JSON shape and types — it can still return a semantically wrong, internally inconsistent value like this.

Correct answer: D. Schemas (via tool_use) guarantee syntactic validity — correct shape and types — but say nothing about whether the content is factually or arithmetically consistent; that needs a separate validation step.

Q10. A timesheet parser uses tool_use with a strict JSON schema, and output is always valid JSON. Sometimes it still produces a total_hours field that doesn't match the sum of the individual daily entries. What does this reveal about schema-constrained output?

A. This means the schema itself is malformed and must be rejected by the API.

B. This only happens when tool_use is used without any temperature setting.

C. ✓ It only guarantees valid JSON shape and types — it can still return a semantically wrong, internally inconsistent value like this.

D. It's impossible for a schema-constrained response to contain this kind of error.

Correct answer: C. Schemas (via tool_use) guarantee syntactic validity — correct shape and types — but say nothing about whether the content is factually or arithmetically consistent; that needs a separate validation step.

Q11. How should a schema handle a signature_date field, when roughly 20% of source documents have no signature at all?

A. Make it required but add a long comment asking the model to guess carefully

B. ✓ type: ["string", "null"] (nullable) rather than required

C. Mark it required so the model always fills it in with something

D. Remove the field from the schema entirely to avoid the issue

Correct answer: B. A required field with no guaranteed source data pressures the model to invent a plausible-looking value; a nullable type lets it correctly express "not present" instead.

Q12. How should a schema handle a middle_name field, when many people simply don't have one?

A. ✓ type: ["string", "null"] (nullable) rather than required

B. Mark it required so the model always fills it in with something

C. Remove the field from the schema entirely to avoid the issue

D. Make it required but add a long comment asking the model to guess carefully

Correct answer: A. A required field with no guaranteed source data pressures the model to invent a plausible-looking value; a nullable type lets it correctly express "not present" instead.

Q13. How should a schema handle a discount_code field, when most orders don't use one?

A. Mark it required so the model always fills it in with something

B. Remove the field from the schema entirely to avoid the issue

C. Make it required but add a long comment asking the model to guess carefully

D. ✓ type: ["string", "null"] (nullable) rather than required

Correct answer: D. A required field with no guaranteed source data pressures the model to invent a plausible-looking value; a nullable type lets it correctly express "not present" instead.

Q14. How should a schema handle a co_signer field on a loan form, when most loans have none?

A. Remove the field from the schema entirely to avoid the issue

B. Make it required but add a long comment asking the model to guess carefully

C. ✓ type: ["string", "null"] (nullable) rather than required

D. Mark it required so the model always fills it in with something

Correct answer: C. A required field with no guaranteed source data pressures the model to invent a plausible-looking value; a nullable type lets it correctly express "not present" instead.

Q15. How should a schema handle a return_reason field, when most orders are never returned?

A. Make it required but add a long comment asking the model to guess carefully

B. ✓ type: ["string", "null"] (nullable) rather than required

C. Mark it required so the model always fills it in with something

D. Remove the field from the schema entirely to avoid the issue

Correct answer: B. A required field with no guaranteed source data pressures the model to invent a plausible-looking value; a nullable type lets it correctly express "not present" instead.

Q16. How should a schema handle a secondary_phone field, when most customers only list one number?

A. ✓ type: ["string", "null"] (nullable) rather than required

B. Mark it required so the model always fills it in with something

C. Remove the field from the schema entirely to avoid the issue

D. Make it required but add a long comment asking the model to guess carefully

Correct answer: A. A required field with no guaranteed source data pressures the model to invent a plausible-looking value; a nullable type lets it correctly express "not present" instead.

Q17. How should a schema handle a cancellation_date field, when most subscriptions are still active?

A. Mark it required so the model always fills it in with something

B. Remove the field from the schema entirely to avoid the issue

C. Make it required but add a long comment asking the model to guess carefully

D. ✓ type: ["string", "null"] (nullable) rather than required

Correct answer: D. A required field with no guaranteed source data pressures the model to invent a plausible-looking value; a nullable type lets it correctly express "not present" instead.

Q18. How should a schema handle a warranty_claim_id field, when most products have never had a claim?

A. Remove the field from the schema entirely to avoid the issue

B. Make it required but add a long comment asking the model to guess carefully

C. ✓ type: ["string", "null"] (nullable) rather than required

D. Mark it required so the model always fills it in with something

Correct answer: C. A required field with no guaranteed source data pressures the model to invent a plausible-looking value; a nullable type lets it correctly express "not present" instead.

Q19. A fixed-category enum for customer support ticket category classification doesn't cover every real input the system sees. What should be added?

A. Make the field free text instead, discarding structure entirely.

B. ✓ Add explicit "unclear" and/or "other" values to the enum so genuinely ambiguous or novel inputs have a truthful place to land.

C. Force every input into the nearest existing category no matter how poor the fit.

D. Leave the enum as-is; mismatches are rare enough to ignore.

Correct answer: B. Enums without an escape-hatch value force bad fits into the nearest wrong category, silently corrupting the data; "unclear"/"other" values let the model express genuine ambiguity truthfully.

Q20. A fixed-category enum for news article topic classification doesn't cover every real input the system sees. What should be added?

A. ✓ Add explicit "unclear" and/or "other" values to the enum so genuinely ambiguous or novel inputs have a truthful place to land.

B. Force every input into the nearest existing category no matter how poor the fit.

C. Leave the enum as-is; mismatches are rare enough to ignore.

D. Make the field free text instead, discarding structure entirely.

Correct answer: A. Enums without an escape-hatch value force bad fits into the nearest wrong category, silently corrupting the data; "unclear"/"other" values let the model express genuine ambiguity truthfully.

Q21. A fixed-category enum for product defect type classification doesn't cover every real input the system sees. What should be added?

A. Force every input into the nearest existing category no matter how poor the fit.

B. Leave the enum as-is; mismatches are rare enough to ignore.

C. Make the field free text instead, discarding structure entirely.

D. ✓ Add explicit "unclear" and/or "other" values to the enum so genuinely ambiguous or novel inputs have a truthful place to land.

Correct answer: D. Enums without an escape-hatch value force bad fits into the nearest wrong category, silently corrupting the data; "unclear"/"other" values let the model express genuine ambiguity truthfully.

Q22. A fixed-category enum for job application status classification doesn't cover every real input the system sees. What should be added?

A. Leave the enum as-is; mismatches are rare enough to ignore.

B. Make the field free text instead, discarding structure entirely.

C. ✓ Add explicit "unclear" and/or "other" values to the enum so genuinely ambiguous or novel inputs have a truthful place to land.

D. Force every input into the nearest existing category no matter how poor the fit.

Correct answer: C. Enums without an escape-hatch value force bad fits into the nearest wrong category, silently corrupting the data; "unclear"/"other" values let the model express genuine ambiguity truthfully.

Q23. A fixed-category enum for expense category classification doesn't cover every real input the system sees. What should be added?

A. Make the field free text instead, discarding structure entirely.

B. ✓ Add explicit "unclear" and/or "other" values to the enum so genuinely ambiguous or novel inputs have a truthful place to land.

C. Force every input into the nearest existing category no matter how poor the fit.

D. Leave the enum as-is; mismatches are rare enough to ignore.

Correct answer: B. Enums without an escape-hatch value force bad fits into the nearest wrong category, silently corrupting the data; "unclear"/"other" values let the model express genuine ambiguity truthfully.

Q24. A fixed-category enum for support-call outcome classification doesn't cover every real input the system sees. What should be added?

A. ✓ Add explicit "unclear" and/or "other" values to the enum so genuinely ambiguous or novel inputs have a truthful place to land.

B. Force every input into the nearest existing category no matter how poor the fit.

C. Leave the enum as-is; mismatches are rare enough to ignore.

D. Make the field free text instead, discarding structure entirely.

Correct answer: A. Enums without an escape-hatch value force bad fits into the nearest wrong category, silently corrupting the data; "unclear"/"other" values let the model express genuine ambiguity truthfully.

Q25. A classification prompt already handles clearly positive and clearly negative reviews reliably. Where should additional few-shot examples be focused for the best improvement?

A. Target the few-shot examples at clearly positive and clearly negative reviews, since those are the most common cases.

B. Use no few-shot examples at all and rely purely on the instructions.

C. Use the same handful of generic examples regardless of what the model struggles with.

D. ✓ Target the few-shot examples at ambiguous, mixed-sentiment reviews, since that's where the model is actually likely to err.

Correct answer: D. Few-shot examples are most valuable when targeted at the model's actual failure cases — ambiguous or boundary inputs — not at cases it already handles well.

Q26. A classification prompt already handles obviously spam and obviously legitimate emails reliably. Where should additional few-shot examples be focused for the best improvement?

A. Use no few-shot examples at all and rely purely on the instructions.

B. Use the same handful of generic examples regardless of what the model struggles with.

C. ✓ Target the few-shot examples at borderline promotional emails that could go either way, since that's where the model is actually likely to err.

D. Target the few-shot examples at obviously spam and obviously legitimate emails, since those are the most common cases.

Correct answer: C. Few-shot examples are most valuable when targeted at the model's actual failure cases — ambiguous or boundary inputs — not at cases it already handles well.

Q27. A classification prompt already handles clearly urgent and clearly non-urgent support tickets reliably. Where should additional few-shot examples be focused for the best improvement?

A. Use the same handful of generic examples regardless of what the model struggles with.

B. ✓ Target the few-shot examples at tickets with mixed urgency signals, since that's where the model is actually likely to err.

C. Target the few-shot examples at clearly urgent and clearly non-urgent support tickets, since those are the most common cases.

D. Use no few-shot examples at all and rely purely on the instructions.

Correct answer: B. Few-shot examples are most valuable when targeted at the model's actual failure cases — ambiguous or boundary inputs — not at cases it already handles well.

Q28. A classification prompt already handles clearly in-scope and clearly out-of-scope requests reliably. Where should additional few-shot examples be focused for the best improvement?

A. ✓ Target the few-shot examples at requests that sit right at the scope boundary, since that's where the model is actually likely to err.

B. Target the few-shot examples at clearly in-scope and clearly out-of-scope requests, since those are the most common cases.

C. Use no few-shot examples at all and rely purely on the instructions.

D. Use the same handful of generic examples regardless of what the model struggles with.

Correct answer: A. Few-shot examples are most valuable when targeted at the model's actual failure cases — ambiguous or boundary inputs — not at cases it already handles well.

Q29. A classification prompt already handles high-confidence and low-confidence transcription segments reliably. Where should additional few-shot examples be focused for the best improvement?

A. Target the few-shot examples at high-confidence and low-confidence transcription segments, since those are the most common cases.

B. Use no few-shot examples at all and rely purely on the instructions.

C. Use the same handful of generic examples regardless of what the model struggles with.

D. ✓ Target the few-shot examples at segments with background noise or overlapping speech, since that's where the model is actually likely to err.

Correct answer: D. Few-shot examples are most valuable when targeted at the model's actual failure cases — ambiguous or boundary inputs — not at cases it already handles well.

Q30. A classification prompt already handles simple single-item orders and simple bulk orders reliably. Where should additional few-shot examples be focused for the best improvement?

A. Use no few-shot examples at all and rely purely on the instructions.

B. Use the same handful of generic examples regardless of what the model struggles with.

C. ✓ Target the few-shot examples at orders that mix multiple order types in one submission, since that's where the model is actually likely to err.

D. Target the few-shot examples at simple single-item orders and simple bulk orders, since those are the most common cases.

Correct answer: C. Few-shot examples are most valuable when targeted at the model's actual failure cases — ambiguous or boundary inputs — not at cases it already handles well.

Q31. A classification prompt already handles clean, well-formatted input documents reliably. Where should additional few-shot examples be focused for the best improvement?

A. Use the same handful of generic examples regardless of what the model struggles with.

B. ✓ Target the few-shot examples at malformed or partially corrupted input documents, since that's where the model is actually likely to err.

C. Target the few-shot examples at clean, well-formatted input documents, since those are the most common cases.

D. Use no few-shot examples at all and rely purely on the instructions.

Correct answer: B. Few-shot examples are most valuable when targeted at the model's actual failure cases — ambiguous or boundary inputs — not at cases it already handles well.

Q32. A classification prompt already handles single-language text samples reliably. Where should additional few-shot examples be focused for the best improvement?

A. ✓ Target the few-shot examples at code-switched or multi-language text samples, since that's where the model is actually likely to err.

B. Target the few-shot examples at single-language text samples, since those are the most common cases.

C. Use no few-shot examples at all and rely purely on the instructions.

D. Use the same handful of generic examples regardless of what the model struggles with.

Correct answer: A. Few-shot examples are most valuable when targeted at the model's actual failure cases — ambiguous or boundary inputs — not at cases it already handles well.

Q33. A retry-with-feedback loop is being considered for a total that was miscalculated from correctly-extracted line items. Will retrying likely fix it?

A. No — retrying never helps with extraction errors.

B. Retry-with-feedback is only useful for formatting issues, never content issues.

C. Whether retrying helps depends solely on the model's temperature setting.

D. ✓ Yes — the correct information exists in the input; retrying with feedback can recover it.

Correct answer: D. Retry-with-feedback works when the correct data exists in the input but was mis-extracted or miscalculated; it structurally cannot recover information that was never present in the source.

Q34. A retry-with-feedback loop is being considered for a field the source document never actually contained. Will retrying likely fix it?

A. Retry-with-feedback is only useful for formatting issues, never content issues.

B. Whether retrying helps depends solely on the model's temperature setting.

C. ✓ No — the information isn't in the source; retrying can't recover data that was never provided.

D. Yes — enough retries will eventually surface the missing information.

Correct answer: C. Retry-with-feedback works when the correct data exists in the input but was mis-extracted or miscalculated; it structurally cannot recover information that was never present in the source.

Q35. A retry-with-feedback loop is being considered for a date that was misread due to an ambiguous format in the source text. Will retrying likely fix it?

A. Whether retrying helps depends solely on the model's temperature setting.
B. ✓ Yes — the correct information exists in the input; retrying with feedback can recover it.
C. No — retrying never helps with extraction errors.
D. Retry-with-feedback is only useful for formatting issues, never content issues.

Correct answer: B. Retry-with-feedback works when the correct data exists in the input but was mis-extracted or miscalculated; it structurally cannot recover information that was never present in the source.

Q36. A retry-with-feedback loop is being considered for a customer's stated preference that was never mentioned anywhere in the input. Will retrying likely fix it?

A. ✓ No — the information isn't in the source; retrying can't recover data that was never provided.
B. Yes — enough retries will eventually surface the missing information.
C. Retry-with-feedback is only useful for formatting issues, never content issues.
D. Whether retrying helps depends solely on the model's temperature setting.

Correct answer: A. Retry-with-feedback works when the correct data exists in the input but was mis-extracted or miscalculated; it structurally cannot recover information that was never present in the source.

Q37. A retry-with-feedback loop is being considered for a category that was misclassified despite clear signal in the text. Will retrying likely fix it?

A. No — retrying never helps with extraction errors.
B. Retry-with-feedback is only useful for formatting issues, never content issues.
C. Whether retrying helps depends solely on the model's temperature setting.
D. ✓ Yes — the correct information exists in the input; retrying with feedback can recover it.

Correct answer: D. Retry-with-feedback works when the correct data exists in the input but was mis-extracted or miscalculated; it structurally cannot recover information that was never present in the source.

Q38. A retry-with-feedback loop is being considered for a specific figure that simply isn't present anywhere in the source document. Will retrying likely fix it?

A. Retry-with-feedback is only useful for formatting issues, never content issues.
B. Whether retrying helps depends solely on the model's temperature setting.
C. ✓ No — the information isn't in the source; retrying can't recover data that was never provided.
D. Yes — enough retries will eventually surface the missing information.

Correct answer: C. Retry-with-feedback works when the correct data exists in the input but was mis-extracted or miscalculated; it structurally cannot recover information that was never present in the source.

Q39. A retry-with-feedback loop is being considered for a name that was mis-parsed from a clearly-formatted input field. Will retrying likely fix it?

A. Whether retrying helps depends solely on the model's temperature setting.
B. ✓ Yes — the correct information exists in the input; retrying with feedback can recover it.
C. No — retrying never helps with extraction errors.
D. Retry-with-feedback is only useful for formatting issues, never content issues.

Correct answer: B. Retry-with-feedback works when the correct data exists in the input but was mis-extracted or miscalculated; it structurally cannot recover information that was never present in the source.

Q40. A retry-with-feedback loop is being considered for a required detail the user never provided in their original request. Will retrying likely fix it?

A. ✓ No — the information isn't in the source; retrying can't recover data that was never provided.
B. Yes — enough retries will eventually surface the missing information.
C. Retry-with-feedback is only useful for formatting issues, never content issues.
D. Whether retrying helps depends solely on the model's temperature setting.

Correct answer: A. Retry-with-feedback works when the correct data exists in the input but was mis-extracted or miscalculated; it structurally cannot recover information that was never present in the source.

Q41. Is the Batch API (roughly 50% cheaper, up to a 24-hour window, no latency SLA) a good fit for an overnight bulk reclassification of a year of support tickets?

A. Poor fit — the Batch API is never appropriate for bulk work.
B. Batch API suitability depends only on document length, not on latency needs.
C. This should always run synchronously regardless of the workload's nature.
D. ✓ Good fit — non-blocking, no one waiting in real time, tolerant of the up-to-24-hour window.

Correct answer: D. Batch API fit hinges on whether anyone is actively blocked waiting on the result — non-blocking, tolerant workloads are a good fit; latency-sensitive, blocking workloads are not.

Q42. Is the Batch API (roughly 50% cheaper, up to a 24-hour window, no latency SLA) a good fit for a blocking pre-merge CI check a developer is actively waiting on?

A. Batch API suitability depends only on document length, not on latency needs.
B. This should always run synchronously regardless of the workload's nature.
C. ✓ Poor fit — latency-sensitive with someone actively waiting; the Batch API offers no latency SLA.
D. Good fit — cost savings always outweigh latency concerns.

Correct answer: C. Batch API fit hinges on whether anyone is actively blocked waiting on the result — non-blocking, tolerant workloads are a good fit; latency-sensitive, blocking workloads are not.

Q43. Is the Batch API (roughly 50% cheaper, up to a 24-hour window, no latency SLA) a good fit for a nightly technical-debt summary report?

A. This should always run synchronously regardless of the workload's nature.
B. ✓ Good fit — non-blocking, no one waiting in real time, tolerant of the up-to-24-hour window.
C. Poor fit — the Batch API is never appropriate for bulk work.
D. Batch API suitability depends only on document length, not on latency needs.

Correct answer: B. Batch API fit hinges on whether anyone is actively blocked waiting on the result — non-blocking, tolerant workloads are a good fit; latency-sensitive, blocking workloads are not.

Q44. Is the Batch API (roughly 50% cheaper, up to a 24-hour window, no latency SLA) a good fit for a real-time chat support interaction?

A. ✓ Poor fit — latency-sensitive with someone actively waiting; the Batch API offers no latency SLA.
B. Good fit — cost savings always outweigh latency concerns.
C. Batch API suitability depends only on document length, not on latency needs.
D. This should always run synchronously regardless of the workload's nature.

Correct answer: A. Batch API fit hinges on whether anyone is actively blocked waiting on the result — non-blocking, tolerant workloads are a good fit; latency-sensitive, blocking workloads are not.

Q45. Is the Batch API (roughly 50% cheaper, up to a 24-hour window, no latency SLA) a good fit for a large one-time migration of historical records to a new schema?

A. Poor fit — the Batch API is never appropriate for bulk work.
B. Batch API suitability depends only on document length, not on latency needs.
C. This should always run synchronously regardless of the workload's nature.
D. ✓ Good fit — non-blocking, no one waiting in real time, tolerant of the up-to-24-hour window.

Correct answer: D. Batch API fit hinges on whether anyone is actively blocked waiting on the result — non-blocking, tolerant workloads are a good fit; latency-sensitive, blocking workloads are not.

Q46. Is the Batch API (roughly 50% cheaper, up to a 24-hour window, no latency SLA) a good fit for a live user-facing search-autocomplete feature?

A. Batch API suitability depends only on document length, not on latency needs.
B. This should always run synchronously regardless of the workload's nature.
C. ✓ Poor fit — latency-sensitive with someone actively waiting; the Batch API offers no latency SLA.
D. Good fit — cost savings always outweigh latency concerns.

Correct answer: C. Batch API fit hinges on whether anyone is actively blocked waiting on the result — non-blocking, tolerant workloads are a good fit; latency-sensitive, blocking workloads are not.

Q47. Is the Batch API (roughly 50% cheaper, up to a 24-hour window, no latency SLA) a good fit for a weekly analytics rollup with no user waiting on it?

A. This should always run synchronously regardless of the workload's nature.

B. ✓ Good fit — non-blocking, no one waiting in real time, tolerant of the up-to-24-hour window.

C. Poor fit — the Batch API is never appropriate for bulk work.

D. Batch API suitability depends only on document length, not on latency needs.

Correct answer: B. Batch API fit hinges on whether anyone is actively blocked waiting on the result — non-blocking, tolerant workloads are a good fit; latency-sensitive, blocking workloads are not.

Q48. Is the Batch API (roughly 50% cheaper, up to a 24-hour window, no latency SLA) a good fit for an interactive code-review comment a developer expects instantly?

A. ✓ Poor fit — latency-sensitive with someone actively waiting; the Batch API offers no latency SLA.

B. Good fit — cost savings always outweigh latency concerns.

C. Batch API suitability depends only on document length, not on latency needs.

D. This should always run synchronously regardless of the workload's nature.

Correct answer: A. Batch API fit hinges on whether anyone is actively blocked waiting on the result — non-blocking, tolerant workloads are a good fit; latency-sensitive, blocking workloads are not.

Q49. Is the Batch API (roughly 50% cheaper, up to a 24-hour window, no latency SLA) a good fit for a bulk overnight re-scoring of a large document archive?

A. Poor fit — the Batch API is never appropriate for bulk work.

B. Batch API suitability depends only on document length, not on latency needs.

C. This should always run synchronously regardless of the workload's nature.

D. ✓ Good fit — non-blocking, no one waiting in real time, tolerant of the up-to-24-hour window.

Correct answer: D. Batch API fit hinges on whether anyone is actively blocked waiting on the result — non-blocking, tolerant workloads are a good fit; latency-sensitive, blocking workloads are not.

Q50. Is the Batch API (roughly 50% cheaper, up to a 24-hour window, no latency SLA) a good fit for a synchronous fraud-check that must complete before a transaction clears?

A. Batch API suitability depends only on document length, not on latency needs.

B. This should always run synchronously regardless of the workload's nature.

C. ✓ Poor fit — latency-sensitive with someone actively waiting; the Batch API offers no latency SLA.

D. Good fit — cost savings always outweigh latency concerns.

Correct answer: C. Batch API fit hinges on whether anyone is actively blocked waiting on the result — non-blocking, tolerant workloads are a good fit; latency-sensitive, blocking workloads are not.

Q51. A system needs to reliably extract 5 structured fields from unstructured customer emails at scale. What's the most robust approach?

A. Avoid structured extraction altogether and read the response manually.

B. ✓ Use tool_use with a defined JSON schema so the API enforces valid structure directly.

C. Ask the model in free text to "please respond in JSON" and parse the string yourself.

D. Use free text and apply a regex to extract fields after the fact.

Correct answer: B. tool_use with a schema is enforced by the API itself, eliminating malformed-JSON failures that free-text-plus-parsing approaches are prone to.

Q52. A system needs to reliably extract 8 structured fields from unstructured scanned invoices at scale. What's the most robust approach?

A. ✓ Use tool_use with a defined JSON schema so the API enforces valid structure directly.

B. Ask the model in free text to "please respond in JSON" and parse the string yourself.

C. Use free text and apply a regex to extract fields after the fact.

D. Avoid structured extraction altogether and read the response manually.

Correct answer: A. tool_use with a schema is enforced by the API itself, eliminating malformed-JSON failures that free-text-plus-parsing approaches are prone to.

Q53. A system needs to reliably extract 6 structured fields from unstructured support transcripts at scale. What's the most robust approach?

A. Ask the model in free text to "please respond in JSON" and parse the string yourself.
B. Use free text and apply a regex to extract fields after the fact.
C. Avoid structured extraction altogether and read the response manually.
D. ✓ Use tool_use with a defined JSON schema so the API enforces valid structure directly.

Correct answer: D. tool_use with a schema is enforced by the API itself, eliminating malformed-JSON failures that free-text-plus-parsing approaches are prone to.

Q54. A system needs to reliably extract 4 structured fields from unstructured job postings at scale. What's the most robust approach?

A. Use free text and apply a regex to extract fields after the fact.
B. Avoid structured extraction altogether and read the response manually.
C. ✓ Use tool_use with a defined JSON schema so the API enforces valid structure directly.
D. Ask the model in free text to "please respond in JSON" and parse the string yourself.

Correct answer: C. tool_use with a schema is enforced by the API itself, eliminating malformed-JSON failures that free-text-plus-parsing approaches are prone to.

Q55. A classification pipeline needs to produce consistent, repeatable structured output across near-identical inputs. How should generation settings be tuned?

A. Randomize temperature per request to average out errors.
B. ✓ Use a lower, more deterministic setting — structured extraction benefits from consistency over creative variation.
C. Use a high temperature to maximize output diversity.
D. Temperature has no effect on structured-output tasks.

Correct answer: B. Deterministic, structured tasks generally benefit from lower temperature settings, since consistency matters more than creative variation for extraction or classification.

Q56. A field-extraction pipeline needs to produce consistent, repeatable structured output across near-identical inputs. How should generation settings be tuned?

A. ✓ Use a lower, more deterministic setting — structured extraction benefits from consistency over creative variation.
B. Use a high temperature to maximize output diversity.
C. Temperature has no effect on structured-output tasks.
D. Randomize temperature per request to average out errors.

Correct answer: A. Deterministic, structured tasks generally benefit from lower temperature settings, since consistency matters more than creative variation for extraction or classification.

Q57. A categorization service needs to produce consistent, repeatable structured output across near-identical inputs. How should generation settings be tuned?

A. Use a high temperature to maximize output diversity.
B. Temperature has no effect on structured-output tasks.
C. Randomize temperature per request to average out errors.
D. ✓ Use a lower, more deterministic setting — structured extraction benefits from consistency over creative variation.

Correct answer: D. Deterministic, structured tasks generally benefit from lower temperature settings, since consistency matters more than creative variation for extraction or classification.

Q58. A data-normalization pipeline needs to produce consistent, repeatable structured output across near-identical inputs. How should generation settings be tuned?

A. Temperature has no effect on structured-output tasks.
B. Randomize temperature per request to average out errors.
C. ✓ Use a lower, more deterministic setting — structured extraction benefits from consistency over creative variation.
D. Use a high temperature to maximize output diversity.

Correct answer: C. Deterministic, structured tasks generally benefit from lower temperature settings, since consistency matters more than creative variation for extraction or classification.

Q59. A document Q&A system re-sends a large, mostly-unchanged reference document on every one of many calls, driving up cost and latency. What should be optimized?

A. Move the large context into the tool schema instead.

B. ✓ Use prompt caching for the large, stable portion of the context that's reused across many calls.

C. Resend the full large context uncached on every single call.

D. Truncate the large context arbitrarily to save cost.

Correct answer: B. Prompt caching lets a large, stable prefix be reused across calls instead of being re-processed from scratch each time, cutting both cost and latency.

Q60. A codebase assistant re-sends a large, mostly-unchanged repository context on every one of many calls, driving up cost and latency. What should be optimized?

A. ✓ Use prompt caching for the large, stable portion of the context that's reused across many calls.

B. Resend the full large context uncached on every single call.

C. Truncate the large context arbitrarily to save cost.

D. Move the large context into the tool schema instead.

Correct answer: A. Prompt caching lets a large, stable prefix be reused across calls instead of being re-processed from scratch each time, cutting both cost and latency.

Q61. A policy chatbot re-sends a large, mostly-unchanged policy manual on every one of many calls, driving up cost and latency. What should be optimized?

A. Resend the full large context uncached on every single call.

B. Truncate the large context arbitrarily to save cost.

C. Move the large context into the tool schema instead.

D. ✓ Use prompt caching for the large, stable portion of the context that's reused across many calls.

Correct answer: D. Prompt caching lets a large, stable prefix be reused across calls instead of being re-processed from scratch each time, cutting both cost and latency.

Q62. A support bot re-sends a large, mostly-unchanged product knowledge base on every one of many calls, driving up cost and latency. What should be optimized?

A. Truncate the large context arbitrarily to save cost.

B. Move the large context into the tool schema instead.

C. ✓ Use prompt caching for the large, stable portion of the context that's reused across many calls.

D. Resend the full large context uncached on every single call.

Correct answer: C. Prompt caching lets a large, stable prefix be reused across calls instead of being re-processed from scratch each time, cutting both cost and latency.

Q63. A prompt for a legal-review assistant mixes instructions, reference clauses, and the actual input together with no clear separation, and the model sometimes confuses one for another. What's the fix?

A. Put all instructions inside the few-shot examples instead of separating them.

B. ✓ Wrap distinct sections (instructions, context, examples, input) in clear structural tags so the model can distinguish them reliably.

C. Concatenate everything into one undifferentiated block of text.

D. Rely on line breaks alone with no labeling.

Correct answer: B. Clear structural separation (e.g., tagged sections) helps the model reliably distinguish instructions from context from the actual input, reducing confusion between them.

Q64. A prompt for a code-review assistant mixes instructions, reference diffs, and the actual input together with no clear separation, and the model sometimes confuses one for another. What's the fix?

A. ✓ Wrap distinct sections (instructions, context, examples, input) in clear structural tags so the model can distinguish them reliably.

B. Concatenate everything into one undifferentiated block of text.

C. Rely on line breaks alone with no labeling.

D. Put all instructions inside the few-shot examples instead of separating them.

Correct answer: A. Clear structural separation (e.g., tagged sections) helps the model reliably distinguish instructions from context from the actual input, reducing confusion between them.

Q65. A prompt for a research summarizer mixes instructions, reference sources, and the actual input together with no clear separation, and the model sometimes confuses one for another. What's the fix?

A. Concatenate everything into one undifferentiated block of text.

B. Rely on line breaks alone with no labeling.

C. Put all instructions inside the few-shot examples instead of separating them.

D. ✓ Wrap distinct sections (instructions, context, examples, input) in clear structural tags so the model can distinguish them reliably.

Correct answer: D. Clear structural separation (e.g., tagged sections) helps the model reliably distinguish instructions from context from the actual input, reducing confusion between them.

Q66. A prompt for a support assistant mixes instructions, reference policy text, and the actual input together with no clear separation, and the model sometimes confuses one for another. What's the fix?

A. Rely on line breaks alone with no labeling.

B. Put all instructions inside the few-shot examples instead of separating them.

C. ✓ Wrap distinct sections (instructions, context, examples, input) in clear structural tags so the model can distinguish them reliably.

D. Concatenate everything into one undifferentiated block of text.

Correct answer: C. Clear structural separation (e.g., tagged sections) helps the model reliably distinguish instructions from context from the actual input, reducing confusion between them.

Q67. A research summarizer occasionally produces claims that sound plausible but aren't actually supported by the provided source documents. What structural change reduces this?

A. Lower the number of source documents provided, to reduce confusion.

B. ✓ Require the model to ground each claim in a specific, quoted or referenced piece of the source material.

C. Ask the model to simply "be more careful" about accuracy.

D. Increase the schema's field count to force more detail.

Correct answer: B. Requiring explicit grounding — tying each claim to a specific source reference — gives the model a concrete anchor and makes ungrounded fabrication easier to catch.

Q68. A legal-analysis tool occasionally produces claims that sound plausible but aren't actually supported by the provided contract text. What structural change reduces this?

A. ✓ Require the model to ground each claim in a specific, quoted or referenced piece of the source material.

B. Ask the model to simply "be more careful" about accuracy.

C. Increase the schema's field count to force more detail.

D. Lower the number of source documents provided, to reduce confusion.

Correct answer: A. Requiring explicit grounding — tying each claim to a specific source reference — gives the model a concrete anchor and makes ungrounded fabrication easier to catch.

Q69. A medical-literature assistant occasionally produces claims that sound plausible but aren't actually supported by the provided cited studies. What structural change reduces this?

A. Ask the model to simply "be more careful" about accuracy.

B. Increase the schema's field count to force more detail.

C. Lower the number of source documents provided, to reduce confusion.

D. ✓ Require the model to ground each claim in a specific, quoted or referenced piece of the source material.

Correct answer: D. Requiring explicit grounding — tying each claim to a specific source reference — gives the model a concrete anchor and makes ungrounded fabrication easier to catch.

Q70. A financial analyst tool occasionally produces claims that sound plausible but aren't actually supported by the provided filings. What structural change reduces this?

A. Increase the schema's field count to force more detail.

B. Lower the number of source documents provided, to reduce confusion.

C. ✓ Require the model to ground each claim in a specific, quoted or referenced piece of the source material.

D. Ask the model to simply "be more careful" about accuracy.

Correct answer: C. Requiring explicit grounding — tying each claim to a specific source reference — gives the model a concrete anchor and makes ungrounded fabrication easier to catch.

Q71. A support bot repeats the same tone-of-voice rule in every single user message instead of setting it once. What's the more maintainable structure?

A. Randomly split instructions between system and user turns.

B. ✓ Put stable, task-wide behavior in the system prompt and per-request specifics in the user turn.

C. Put everything, including the specific request, into the system prompt.

D. Put stable behavioral rules into the user turn on every single call.

Correct answer: B. Stable, task-wide behavior belongs in the system prompt (set once); per-request specifics belong in the user turn — this avoids redundant repetition and keeps the two concerns cleanly separated.

Q72. A classification service repeats the same output-format rule in every single user message instead of setting it once. What's the more maintainable structure?

A. ✓ Put stable, task-wide behavior in the system prompt and per-request specifics in the user turn.

B. Put everything, including the specific request, into the system prompt.

C. Put stable behavioral rules into the user turn on every single call.

D. Randomly split instructions between system and user turns.

Correct answer: A. Stable, task-wide behavior belongs in the system prompt (set once); per-request specifics belong in the user turn — this avoids redundant repetition and keeps the two concerns cleanly separated.

Q73. A code-review bot repeats the same review-standard rule in every single user message instead of setting it once. What's the more maintainable structure?

A. Put everything, including the specific request, into the system prompt.

B. Put stable behavioral rules into the user turn on every single call.

C. Randomly split instructions between system and user turns.

D. ✓ Put stable, task-wide behavior in the system prompt and per-request specifics in the user turn.

Correct answer: D. Stable, task-wide behavior belongs in the system prompt (set once); per-request specifics belong in the user turn — this avoids redundant repetition and keeps the two concerns cleanly separated.

Q74. A summarizer repeats the same length constraint in every single user message instead of setting it once. What's the more maintainable structure?

A. Put stable behavioral rules into the user turn on every single call.

B. Randomly split instructions between system and user turns.

C. ✓ Put stable, task-wide behavior in the system prompt and per-request specifics in the user turn.

D. Put everything, including the specific request, into the system prompt.

Correct answer: C. Stable, task-wide behavior belongs in the system prompt (set once); per-request specifics belong in the user turn — this avoids redundant repetition and keeps the two concerns cleanly separated.

Q75. A team wants to change how a classification prompt is structured but isn't sure if the change will help or hurt accuracy. What should happen before shipping it?

A. Trust that a prompt "looks right" without measuring actual output.

B. ✓ Build a held-out evaluation set with known-correct answers and measure accuracy before shipping a prompt change.

C. Ship the new prompt directly to production and monitor for complaints.

D. Test the prompt once manually on a single example and move on.

Correct answer: B. A held-out evaluation set with known-correct answers lets you measure the actual effect of a prompt change before it reaches production, rather than guessing from how the prompt reads.

Q76. A team wants to change how a extraction prompt is structured but isn't sure if the change will help or hurt accuracy. What should happen before shipping it?

A. ✓ Build a held-out evaluation set with known-correct answers and measure accuracy before shipping a prompt change.

B. Ship the new prompt directly to production and monitor for complaints.

C. Test the prompt once manually on a single example and move on.

D. Trust that a prompt "looks right" without measuring actual output.

Correct answer: A. A held-out evaluation set with known-correct answers lets you measure the actual effect of a prompt change before it reaches production, rather than guessing from how the prompt reads.

Q77. A team wants to change how a summarization prompt is structured but isn't sure if the change will help or hurt accuracy. What should happen before shipping it?

A. Ship the new prompt directly to production and monitor for complaints.

B. Test the prompt once manually on a single example and move on.

C. Trust that a prompt "looks right" without measuring actual output.

D. ✓ Build a held-out evaluation set with known-correct answers and measure accuracy before shipping a prompt change.

Correct answer: D. A held-out evaluation set with known-correct answers lets you measure the actual effect of a prompt change before it reaches production, rather than guessing from how the prompt reads.

Q78. A team wants to change how a routing prompt is structured but isn't sure if the change will help or hurt accuracy. What should happen before shipping it?

A. Test the prompt once manually on a single example and move on.

B. Trust that a prompt "looks right" without measuring actual output.

C. ✓ Build a held-out evaluation set with known-correct answers and measure accuracy before shipping a prompt change.

D. Ship the new prompt directly to production and monitor for complaints.

Correct answer: C. A held-out evaluation set with known-correct answers lets you measure the actual effect of a prompt change before it reaches production, rather than guessing from how the prompt reads.

Q79. A real-time extraction UI streams its structured tool_use response to the client, and a naive downstream parser occasionally chokes on incomplete JSON. What's the correct handling?

A. Assume streamed JSON is always complete after a fixed number of tokens.

B. ✓ Wait for the complete structured response (or use a schema-aware streaming parser) rather than acting on a partially-streamed JSON object.

C. Start acting on the JSON object as soon as the first characters arrive.

D. Disable structured output whenever streaming is needed.

Correct answer: B. Partially-streamed JSON isn't valid until the object is complete — downstream consumers need either to wait for completion or use a parser designed for incremental structured streaming.

Q80. A live classification dashboard streams its structured tool_use response to the client, and a naive downstream parser occasionally chokes on incomplete JSON. What's the correct handling?

A. ✓ Wait for the complete structured response (or use a schema-aware streaming parser) rather than acting on a partially-streamed JSON object.

B. Start acting on the JSON object as soon as the first characters arrive.

C. Disable structured output whenever streaming is needed.

D. Assume streamed JSON is always complete after a fixed number of tokens.

Correct answer: A. Partially-streamed JSON isn't valid until the object is complete — downstream consumers need either to wait for completion or use a parser designed for incremental structured streaming.

Q81. A streaming form-fill assistant streams its structured tool_use response to the client, and a naive downstream parser occasionally chokes on incomplete JSON. What's the correct handling?

A. Start acting on the JSON object as soon as the first characters arrive.

B. Disable structured output whenever streaming is needed.

C. Assume streamed JSON is always complete after a fixed number of tokens.

D. ✓ Wait for the complete structured response (or use a schema-aware streaming parser) rather than acting on a partially-streamed JSON object.

Correct answer: D. Partially-streamed JSON isn't valid until the object is complete — downstream consumers need either to wait for completion or use a parser designed for incremental structured streaming.

Q82. A live triage tool streams its structured tool_use response to the client, and a naive downstream parser occasionally chokes on incomplete JSON. What's the correct handling?

A. Disable structured output whenever streaming is needed.

B. Assume streamed JSON is always complete after a fixed number of tokens.

C. ✓ Wait for the complete structured response (or use a schema-aware streaming parser) rather than acting on a partially-streamed JSON object.

D. Start acting on the JSON object as soon as the first characters arrive.

Correct answer: C. Partially-streamed JSON isn't valid until the object is complete — downstream consumers need either to wait for completion or use a parser designed for incremental structured streaming.

Q83. A classification service sends a much longer prompt than the task actually requires, driving up per-call cost with no accuracy benefit. What's the first optimization to try?

A. Leave prompt length unexamined and optimize only the schema instead.

B. ✓ Trim the prompt to only the context genuinely needed for the task, rather than padding it with unused boilerplate.

C. Add more instructions and examples regardless of whether they're needed.

D. Switch to a larger model as the default fix for cost concerns.

Correct answer: B. Trimming unnecessary content from the prompt is usually the highest-leverage, lowest-risk cost optimization — before reaching for a different model or infrastructure change.

Q84. A extraction pipeline sends a much longer prompt than the task actually requires, driving up per-call cost with no accuracy benefit. What's the first optimization to try?

A. ✓ Trim the prompt to only the context genuinely needed for the task, rather than padding it with unused boilerplate.

B. Add more instructions and examples regardless of whether they're needed.

C. Switch to a larger model as the default fix for cost concerns.

D. Leave prompt length unexamined and optimize only the schema instead.

Correct answer: A. Trimming unnecessary content from the prompt is usually the highest-leverage, lowest-risk cost optimization — before reaching for a different model or infrastructure change.

Q85. A summarization tool sends a much longer prompt than the task actually requires, driving up per-call cost with no accuracy benefit. What's the first optimization to try?

A. Add more instructions and examples regardless of whether they're needed.

B. Switch to a larger model as the default fix for cost concerns.

C. Leave prompt length unexamined and optimize only the schema instead.

D. ✓ Trim the prompt to only the context genuinely needed for the task, rather than padding it with unused boilerplate.

Correct answer: D. Trimming unnecessary content from the prompt is usually the highest-leverage, lowest-risk cost optimization — before reaching for a different model or infrastructure change.

Q86. A routing service sends a much longer prompt than the task actually requires, driving up per-call cost with no accuracy benefit. What's the first optimization to try?

A. Switch to a larger model as the default fix for cost concerns.

B. Leave prompt length unexamined and optimize only the schema instead.

C. ✓ Trim the prompt to only the context genuinely needed for the task, rather than padding it with unused boilerplate.

D. Add more instructions and examples regardless of whether they're needed.

Correct answer: C. Trimming unnecessary content from the prompt is usually the highest-leverage, lowest-risk cost optimization — before reaching for a different model or infrastructure change.

Q87. A handwritten-form extractor sometimes extracts a field with genuine uncertainty (e.g., illegible handwriting, ambiguous phrasing). How should this be surfaced?

A. Silently drop uncertain fields from the output with no indication they were omitted.

B. ✓ Surface a confidence or uncertainty signal in the schema itself so downstream systems can flag or route low-confidence extractions for review.

C. Always return the model's best guess with no way to distinguish confident from uncertain output.

D. Reject the entire extraction whenever any single field is uncertain.

Correct answer: B. Building an explicit confidence or uncertainty signal into the schema lets downstream systems route uncertain extractions for human review instead of silently trusting or discarding them.

Q88. A noisy-transcript parser sometimes extracts a field with genuine uncertainty (e.g., illegible handwriting, ambiguous phrasing). How should this be surfaced?

A. ✓ Surface a confidence or uncertainty signal in the schema itself so downstream systems can flag or route low-confidence extractions for review.

B. Always return the model's best guess with no way to distinguish confident from uncertain output.

C. Reject the entire extraction whenever any single field is uncertain.

D. Silently drop uncertain fields from the output with no indication they were omitted.

Correct answer: A. Building an explicit confidence or uncertainty signal into the schema lets downstream systems route uncertain extractions for human review instead of silently trusting or discarding them.

Q89. A scanned-document extractor sometimes extracts a field with genuine uncertainty (e.g., illegible handwriting, ambiguous phrasing). How should this be surfaced?

A. Always return the model's best guess with no way to distinguish confident from uncertain output.

B. Reject the entire extraction whenever any single field is uncertain.

C. Silently drop uncertain fields from the output with no indication they were omitted.

D. ✓ Surface a confidence or uncertainty signal in the schema itself so downstream systems can flag or route low-confidence extractions for review.

Correct answer: D. Building an explicit confidence or uncertainty signal into the schema lets downstream systems route uncertain extractions for human review instead of silently trusting or discarding them.

Q90. A ambiguous-survey parser sometimes extracts a field with genuine uncertainty (e.g., illegible handwriting, ambiguous phrasing). How should this be surfaced?

A. Reject the entire extraction whenever any single field is uncertain.

B. Silently drop uncertain fields from the output with no indication they were omitted.

C. ✓ Surface a confidence or uncertainty signal in the schema itself so downstream systems can flag or route low-confidence extractions for review.

D. Always return the model's best guess with no way to distinguish confident from uncertain output.

Correct answer: C. Building an explicit confidence or uncertainty signal into the schema lets downstream systems route uncertain extractions for human review instead of silently trusting or discarding them.

Q91. A invoice-extraction service's schema needs a breaking change (renaming a field, changing a type) but multiple downstream consumers depend on the current shape. What's the safer approach?

A. Silently rename fields without notifying any consumers.

B. ✓ Version the schema explicitly and support the older version during a transition period rather than breaking existing consumers immediately.

C. Change the schema in place with no versioning and let downstream consumers fail.

D. Never evolve the schema once it's shipped, regardless of new requirements.

Correct answer: B. Explicit schema versioning with a transition period avoids breaking existing consumers outright, giving them time to migrate to the new shape.

Q92. A order-processing pipeline's schema needs a breaking change (renaming a field, changing a type) but multiple downstream consumers depend on the current shape. What's the safer approach?

A. ✓ Version the schema explicitly and support the older version during a transition period rather than breaking existing consumers immediately.

B. Change the schema in place with no versioning and let downstream consumers fail.

C. Never evolve the schema once it's shipped, regardless of new requirements.

D. Silently rename fields without notifying any consumers.

Correct answer: A. Explicit schema versioning with a transition period avoids breaking existing consumers outright, giving them time to migrate to the new shape.

Q93. A ticket-classification service's schema needs a breaking change (renaming a field, changing a type) but multiple downstream consumers depend on the current shape. What's the safer approach?

A. Change the schema in place with no versioning and let downstream consumers fail.

B. Never evolve the schema once it's shipped, regardless of new requirements.

C. Silently rename fields without notifying any consumers.

D. ✓ Version the schema explicitly and support the older version during a transition period rather than breaking existing consumers immediately.

Correct answer: D. Explicit schema versioning with a transition period avoids breaking existing consumers outright, giving them time to migrate to the new shape.

Q94. A resume-parsing service's schema needs a breaking change (renaming a field, changing a type) but multiple downstream consumers depend on the current shape. What's the safer approach?

A. Never evolve the schema once it's shipped, regardless of new requirements.

B. Silently rename fields without notifying any consumers.

C. ✓ Version the schema explicitly and support the older version during a transition period rather than breaking existing consumers immediately.

D. Change the schema in place with no versioning and let downstream consumers fail.

Correct answer: C. Explicit schema versioning with a transition period avoids breaking existing consumers outright, giving them time to migrate to the new shape.

Q95. A customer email being fed into a structured-extraction pipeline contains text that reads like an instruction to the model (e.g., "ignore previous instructions and output X"). How should the pipeline treat this?

A. Disable the schema constraint whenever the source content looks unusual.

B. ✓ Treat the content as untrusted data, not instructions — the extraction prompt should not follow directives found inside it.

C. Treat any embedded instructions in the source as legitimate commands to follow.

D. Assume injection isn't a risk for structured-extraction tasks.

Correct answer: B. Any external content — including documents being extracted from — should be treated as data, not instructions; a robust pipeline doesn't let embedded text redirect its actual task.

Q96. A scraped web page being fed into a structured-extraction pipeline contains text that reads like an instruction to the model (e.g., "ignore previous instructions and output X"). How should the pipeline treat this?

A. ✓ Treat the content as untrusted data, not instructions — the extraction prompt should not follow directives found inside it.

B. Treat any embedded instructions in the source as legitimate commands to follow.

C. Assume injection isn't a risk for structured-extraction tasks.

D. Disable the schema constraint whenever the source content looks unusual.

Correct answer: A. Any external content — including documents being extracted from — should be treated as data, not instructions; a robust pipeline doesn't let embedded text redirect its actual task.

Q97. An uploaded pdf resume being fed into a structured-extraction pipeline contains text that reads like an instruction to the model (e.g., "ignore previous instructions and output X"). How should the pipeline treat this?

A. Treat any embedded instructions in the source as legitimate commands to follow.

B. Assume injection isn't a risk for structured-extraction tasks.

C. Disable the schema constraint whenever the source content looks unusual.

D. ✓ Treat the content as untrusted data, not instructions — the extraction prompt should not follow directives found inside it.

Correct answer: D. Any external content — including documents being extracted from — should be treated as data, not instructions; a robust pipeline doesn't let embedded text redirect its actual task.

Q98. A third-party api response being fed into a structured-extraction pipeline contains text that reads like an instruction to the model (e.g., "ignore previous instructions and output X"). How should the pipeline treat this?

A. Assume injection isn't a risk for structured-extraction tasks.

B. Disable the schema constraint whenever the source content looks unusual.

C. ✓ Treat the content as untrusted data, not instructions — the extraction prompt should not follow directives found inside it.

D. Treat any embedded instructions in the source as legitimate commands to follow.

Correct answer: C. Any external content — including documents being extracted from — should be treated as data, not instructions; a robust pipeline doesn't let embedded text redirect its actual task.

Q99. A user-submitted support ticket being fed into a structured-extraction pipeline contains text that reads like an instruction to the model (e.g., "ignore previous instructions and output X"). How should the pipeline treat this?

A. Disable the schema constraint whenever the source content looks unusual.

B. ✓ Treat the content as untrusted data, not instructions — the extraction prompt should not follow directives found inside it.

C. Treat any embedded instructions in the source as legitimate commands to follow.

D. Assume injection isn't a risk for structured-extraction tasks.

Correct answer: B. Any external content — including documents being extracted from — should be treated as data, not instructions; a robust pipeline doesn't let embedded text redirect its actual task.

Q100. An ocr'd scanned document being fed into a structured-extraction pipeline contains text that reads like an instruction to the model (e.g., "ignore previous instructions and output X"). How should the pipeline treat this?

A. ✓ Treat the content as untrusted data, not instructions — the extraction prompt should not follow directives found inside it.

B. Treat any embedded instructions in the source as legitimate commands to follow.

C. Assume injection isn't a risk for structured-extraction tasks.

D. Disable the schema constraint whenever the source content looks unusual.

Correct answer: A. Any external content — including documents being extracted from — should be treated as data, not instructions; a robust pipeline doesn't let embedded text redirect its actual task.

Q101. For a product-categorization service, should structured output rely on a plain-text instruction to "respond only in JSON," or on tool_use with a schema?

A. There's no meaningful difference; both approaches are equally reliable.

B. Free-text "please respond in JSON" is always more reliable than tool_use.

C. Neither approach matters since all output should be parsed with a permissive regex anyway.

D. ✓ Prefer tool_use with an explicit schema — it's enforced by the API, unlike asking for JSON in free text and hoping the model complies.

Correct answer: D. tool_use with a schema is enforced at the API level, giving much stronger guarantees than a free-text instruction the model might not follow precisely.

Q102. For a sentiment-scoring pipeline, should structured output rely on a plain-text instruction to "respond only in JSON," or on tool_use with a schema?

A. Free-text "please respond in JSON" is always more reliable than tool_use.

B. Neither approach matters since all output should be parsed with a permissive regex anyway.

C. ✓ Prefer tool_use with an explicit schema — it's enforced by the API, unlike asking for JSON in free text and hoping the model complies.

D. There's no meaningful difference; both approaches are equally reliable.

Correct answer: C. tool_use with a schema is enforced at the API level, giving much stronger guarantees than a free-text instruction the model might not follow precisely.

Q103. For a name-entity extraction service, should structured output rely on a plain-text instruction to "respond only in JSON," or on tool_use with a schema?

A. Neither approach matters since all output should be parsed with a permissive regex anyway.

B. ✓ Prefer tool_use with an explicit schema — it's enforced by the API, unlike asking for JSON in free text and hoping the model complies.

C. There's no meaningful difference; both approaches are equally reliable.

D. Free-text "please respond in JSON" is always more reliable than tool_use.

Correct answer: B. tool_use with a schema is enforced at the API level, giving much stronger guarantees than a free-text instruction the model might not follow precisely.

Q104. For an address-normalization service, should structured output rely on a plain-text instruction to "respond only in JSON," or on tool_use with a schema?

A. ✓ Prefer tool_use with an explicit schema — it's enforced by the API, unlike asking for JSON in free text and hoping the model complies.

B. There's no meaningful difference; both approaches are equally reliable.

C. Free-text "please respond in JSON" is always more reliable than tool_use.

D. Neither approach matters since all output should be parsed with a permissive regex anyway.

Correct answer: A. tool_use with a schema is enforced at the API level, giving much stronger guarantees than a free-text instruction the model might not follow precisely.

Q105. For a metadata-tagging pipeline, should structured output rely on a plain-text instruction to "respond only in JSON," or on tool_use with a schema?

A. There's no meaningful difference; both approaches are equally reliable.

B. Free-text "please respond in JSON" is always more reliable than tool_use.

C. Neither approach matters since all output should be parsed with a permissive regex anyway.

D. ✓ Prefer tool_use with an explicit schema — it's enforced by the API, unlike asking for JSON in free text and hoping the model complies.

Correct answer: D. tool_use with a schema is enforced at the API level, giving much stronger guarantees than a free-text instruction the model might not follow precisely.

Q106. For a lead-qualification service, should structured output rely on a plain-text instruction to "respond only in JSON," or on tool_use with a schema?

A. Free-text "please respond in JSON" is always more reliable than tool_use.

B. Neither approach matters since all output should be parsed with a permissive regex anyway.

C. ✓ Prefer tool_use with an explicit schema — it's enforced by the API, unlike asking for JSON in free text and hoping the model complies.

D. There's no meaningful difference; both approaches are equally reliable.

Correct answer: C. tool_use with a schema is enforced at the API level, giving much stronger guarantees than a free-text instruction the model might not follow precisely.

Q107. In a structured-extraction pipeline, a required numeric field comes back as a non-numeric string. What should happen?

A. Disable validation going forward since it caught an inconvenient error.

B. ✓ Reject the output at the validation layer and route it to a retry-with-feedback pass or human review, rather than passing it downstream as-is.

C. Silently coerce the bad value to something plausible and continue.

D. Pass the invalid output downstream unchanged and let a later system catch it.

Correct answer: B. A validation failure should stop bad data from silently propagating — routing it to retry-with-feedback or human review preserves data quality instead of passing a known-bad record downstream.

Q108. In a structured-extraction pipeline, an enum field comes back with a value outside the allowed set. What should happen?

A. ✓ Reject the output at the validation layer and route it to a retry-with-feedback pass or human review, rather than passing it downstream as-is.

B. Silently coerce the bad value to something plausible and continue.

C. Pass the invalid output downstream unchanged and let a later system catch it.

D. Disable validation going forward since it caught an inconvenient error.

Correct answer: A. A validation failure should stop bad data from silently propagating — routing it to retry-with-feedback or human review preserves data quality instead of passing a known-bad record downstream.

Q109. In a structured-extraction pipeline, a date field comes back in an unparseable format. What should happen?

A. Silently coerce the bad value to something plausible and continue.

B. Pass the invalid output downstream unchanged and let a later system catch it.

C. Disable validation going forward since it caught an inconvenient error.

D. ✓ Reject the output at the validation layer and route it to a retry-with-feedback pass or human review, rather than passing it downstream as-is.

Correct answer: D. A validation failure should stop bad data from silently propagating — routing it to retry-with-feedback or human review preserves data quality instead of passing a known-bad record downstream.

Q110. In a structured-extraction pipeline, a nested object is missing a required sub-field. What should happen?

A. Pass the invalid output downstream unchanged and let a later system catch it.

B. Disable validation going forward since it caught an inconvenient error.

C. ✓ Reject the output at the validation layer and route it to a retry-with-feedback pass or human review, rather than passing it downstream as-is.

D. Silently coerce the bad value to something plausible and continue.

Correct answer: C. A validation failure should stop bad data from silently propagating — routing it to retry-with-feedback or human review preserves data quality instead of passing a known-bad record downstream.

Q111. In a structured-extraction pipeline, an array field that should never be empty comes back empty. What should happen?

A. Disable validation going forward since it caught an inconvenient error.

B. ✓ Reject the output at the validation layer and route it to a retry-with-feedback pass or human review, rather than passing it downstream as-is.

C. Silently coerce the bad value to something plausible and continue.

D. Pass the invalid output downstream unchanged and let a later system catch it.

Correct answer: B. A validation failure should stop bad data from silently propagating — routing it to retry-with-feedback or human review preserves data quality instead of passing a known-bad record downstream.

Q112. In a structured-extraction pipeline, a cross-field consistency check fails after extraction. What should happen?

A. ✓ Reject the output at the validation layer and route it to a retry-with-feedback pass or human review, rather than passing it downstream as-is.

B. Silently coerce the bad value to something plausible and continue.

C. Pass the invalid output downstream unchanged and let a later system catch it.

D. Disable validation going forward since it caught an inconvenient error.

Correct answer: A. A validation failure should stop bad data from silently propagating — routing it to retry-with-feedback or human review preserves data quality instead of passing a known-bad record downstream.

Q113. A classification prompt handles customer support tickets that are worded slightly differently but describe the same underlying issue inconsistently, producing different labels for what are effectively equivalent inputs. What's the most direct fix?

A. Assume the model will generalize correctly with zero examples provided.

B. Add more schema fields, since the issue must be structural, not example-related.

C. Lower the model's context window to force more consistent behavior.

D. ✓ Add few-shot examples spanning that surface variation so the model generalizes past superficial wording differences to the underlying pattern.

Correct answer: D. Inconsistent handling of superficially-different-but-substantively-similar inputs is a strong signal that few-shot examples spanning that variation are needed, so the model learns to generalize past wording.

Q114. A classification prompt handles invoices from the same vendor with minor formatting differences month to month inconsistently, producing different labels for what are effectively equivalent inputs. What's the most direct fix?

A. Add more schema fields, since the issue must be structural, not example-related.

B. Lower the model's context window to force more consistent behavior.

C. ✓ Add few-shot examples spanning that surface variation so the model generalizes past superficial wording differences to the underlying pattern.

D. Assume the model will generalize correctly with zero examples provided.

Correct answer: C. Inconsistent handling of superficially-different-but-substantively-similar inputs is a strong signal that few-shot examples spanning that variation are needed, so the model learns to generalize past wording.

Q115. A classification prompt handles job postings for the same role posted on different job boards with different templates inconsistently, producing different labels for what are effectively equivalent inputs. What's the most direct fix?

A. Lower the model's context window to force more consistent behavior.

B. ✓ Add few-shot examples spanning that surface variation so the model generalizes past superficial wording differences to the underlying pattern.

C. Assume the model will generalize correctly with zero examples provided.

D. Add more schema fields, since the issue must be structural, not example-related.

Correct answer: B. Inconsistent handling of superficially-different-but-substantively-similar inputs is a strong signal that few-shot examples spanning that variation are needed, so the model learns to generalize past wording.

Q116. A classification prompt handles product reviews expressing the same sentiment with different phrasing inconsistently, producing different labels for what are effectively equivalent inputs. What's the most direct fix?

A. ✓ Add few-shot examples spanning that surface variation so the model generalizes past superficial wording differences to the underlying pattern.

B. Assume the model will generalize correctly with zero examples provided.

C. Add more schema fields, since the issue must be structural, not example-related.

D. Lower the model's context window to force more consistent behavior.

Correct answer: A. Inconsistent handling of superficially-different-but-substantively-similar inputs is a strong signal that few-shot examples spanning that variation are needed, so the model learns to generalize past wording.

Q117. A classification prompt handles log lines describing the same error with slightly different timestamps and IDs inconsistently, producing different labels for what are effectively equivalent inputs. What's the most direct fix?

A. Assume the model will generalize correctly with zero examples provided.

B. Add more schema fields, since the issue must be structural, not example-related.

C. Lower the model's context window to force more consistent behavior.

D. ✓ Add few-shot examples spanning that surface variation so the model generalizes past superficial wording differences to the underlying pattern.

Correct answer: D. Inconsistent handling of superficially-different-but-substantively-similar inputs is a strong signal that few-shot examples spanning that variation are needed, so the model learns to generalize past wording.

Q118. A classification prompt handles contracts from the same template with minor clause reordering inconsistently, producing different labels for what are effectively equivalent inputs. What's the most direct fix?

A. Add more schema fields, since the issue must be structural, not example-related.

B. Lower the model's context window to force more consistent behavior.

C. ✓ Add few-shot examples spanning that surface variation so the model generalizes past superficial wording differences to the underlying pattern.

D. Assume the model will generalize correctly with zero examples provided.

Correct answer: C. Inconsistent handling of superficially-different-but-substantively-similar inputs is a strong signal that few-shot examples spanning that variation are needed, so the model learns to generalize past wording.

End of Domain 3 deep-dive. This is an independent study aid; verify current exam logistics and blueprint details against your Claude Partner Network portal before sitting the exam.